

Multi-Agent Based Time Critical FaaS Scheduling in Edge-Cloud Continuum

Yitao Lei¹, Mengbing Zhou¹, Paul Daniëls¹, Zhiming Zhao^{1,2*}

¹ *Institute of Informatics, University of Amsterdam, Amsterdam, Netherlands*

² *LifeWatch ERIC, Virtual Lab Innovation Center, Amsterdam, Netherlands*

{yitao.lei@student.uva.nl, m.zhou2@uva.nl, p.e.g.danielse@uva.nl, z.zhao@uva.nl}

* *Corresponding author*

Abstract—Function-as-a-Service (FaaS) platforms are increasingly deployed across heterogeneous edge-cloud environments, where latency-sensitive applications require both low response times and robust scalability. Existing scheduling architectures—centralized, federated, and decentralized—offer different trade-offs between global optimization, wide-area latency, and regional autonomy. However, a single architecture often struggles to fit in dynamic workloads and changing system conditions. This paper proposes an architecture-adaptive, multi-agent scheduling framework for FaaS that can operate under centralized, federated, and decentralized modes and dynamically switch between them at runtime, guided by a QoS-driven evolution mechanism based on long-tail latency signals derived from p95/p50 execution-time ratios. We prototype a framework on OpenFaaS, combining Kubernetes-based cloud deployments with lightweight edge instances on a cloud-hosted edge-cloud testbed. Experimental results show that the dynamic architecture achieves competitive or near-optimal total completion times across workloads, and delivers more stable response-time distributions than purely centralized, federated, or decentralized baselines, while incurring negligible architecture-switching latency and resource overhead.

Index Terms—Serverless Computing, Edge-Cloud Continuum, Multi-Agent Scheduling, Architecture Evolution

I. INTRODUCTION

Function-as-a-Service (FaaS) abstractions have become a popular substrate for event-driven and microservice-based applications, promising fine-grained scalability and simplified operations [1]. However, most existing FaaS platforms were originally designed for centralized cloud environments, whereas emerging latency-sensitive and context-aware applications increasingly require computation to be distributed across the edge-cloud continuum [2]. For example, large-scale digital twin systems must continuously ingest massive event streams and produce timely control feedback across geographically distributed resources under strict real-time requirements [3]. In such settings, FaaS provides a natural execution platform for implementing parts of the event-driven pipeline, but its scheduling mechanisms must simultaneously contend with heterogeneous compute capacities, highly variable network conditions, and stringent quality-of-service (QoS) constraints, especially with respect to end-to-end latency and timeliness [4], [5].

Prior work has explored three main scheduling architectures for edge-cloud systems: centralized, federated, and decentralized. In centralized models, a global controller coordinates

function invocations across the continuum, enabling holistic optimization but risking bottlenecks and higher latency when devices are geographically dispersed [6]. Federated architectures distribute control across regional or zone-level schedulers that cooperate for cross-zone load balancing, improving scalability and locality at the cost of more complex coordination [7]. Decentralized designs push decisions to individual nodes or peer clusters, providing high fault tolerance and local responsiveness but often leading to suboptimal global decisions and additional communication overhead [8]. None of these models is universally optimal: the best choice depends on workload characteristics, network conditions, and QoS requirements. However, static architectural choices cannot maintain efficient and robust behavior as these factors evolve over time.

This raises the question of whether the operations plane should not only adapt function placement, but also reconfigure the scheduling architecture at runtime in heterogeneous and dynamic edge-cloud environments. Under light load and stable network conditions, a decentralized architecture may provide low latency and minimal coordination overhead, whereas under heavy load or wide-area congestion, centralized or federated control may be needed to keep end-to-end response times and deadline miss rates within acceptable bounds. In large-scale digital twin systems, where virtual models must remain closely synchronized with their physical counterparts, even a small fraction of highly delayed updates can destabilize control feedback and violate real-time constraints despite relatively low average latency [3]. However, existing serverless systems and scheduling frameworks typically assume a fixed control structure: they either optimize function placement within a predefined centralized, federated, or decentralized architecture [9]–[11], or apply QoS-aware policies without changing the underlying coordination model [12], [13]. This lack of architecture-level adaptability limits their ability to provide robust QoS across diverse operating conditions.

To address these challenges, this work proposes a modular multi-agent scheduling framework for FaaS in the edge-cloud continuum that can operate under centralized, federated, and decentralized coordination models and dynamically switch between them at runtime based on system behavior. The framework leverages long-tail latency as its primary QoS signal: by monitoring recent execution times and deriving a simple tail ratio (p95/p50), it captures skewness in the latency

distribution that directly correlates with bursty delays, control-loop deadline violations, and widened latency dispersion in digital-twin-style workloads. Rather than targeting average latency, the system treats sustained increases in tail ratio as evidence of emerging performance degradation and adaptively shifts control from decentralized towards more coordinated federated or centralized architectures when necessary, while remaining conservative under stable conditions. We implement this design on OpenFaaS [14] and evaluate it on a cloud-hosted edge-cloud testbed under varying concurrency levels, showing that the dynamic architecture maintains competitive total completion time while keeping latency dispersion and tail amplification within bounded ranges compared to static architectures, with negligible scheduling overhead and comparable resource usage.

The main contributions of this work are as follows:

- **architecture-adaptive, multi-agent scheduling framework:** We design a modular control plane that supports centralized, federated, and decentralized scheduling in the edge-cloud continuum and can dynamically switch between these architectures at runtime.
- **QoS-driven architecture evolution:** We introduce a tail-latency-aware mechanism that monitors p95/p50 ratios and uses lightweight threshold-based policies to adapt the control scope, enabling intelligent and adaptive edge-cloud operations without redesigning the underlying platform.
- **Prototype and evaluation:** We implement the framework on OpenFaaS and evaluate it on a cloud-hosted edge-cloud testbed with emulated RTTs, showing that dynamic architecture switching improves total completion time and stabilizes tail latency compared to static architectures, with negligible decision overhead and similar resource usage.

Overall, this work demonstrates that treating the scheduling architecture itself as a runtime control variable—rather than a fixed design choice—is both practical and beneficial for serverless systems in heterogeneous edge-cloud environments. By combining a multi-agent control plane with tail-aware, QoS-driven adaptation, the proposed approach closes an important gap between fine-grained latency-sensitive placement and coarse-grained architectural flexibility, and opens up opportunities for future multi-objective scheduling that also considers factors such as energy, cost, and resilience.

II. RELATED WORK

Research on serverless scheduling in the edge-cloud continuum has explored diverse control architectures and placement strategies. Our prior empirical study compares centralized, federated, and decentralized offloading architectures for OpenFaaS deployments across edge and cloud nodes, demonstrating that architectural choices critically affect latency and resource efficiency under different workload patterns [15]. The results reveal clear trade-offs among these architectures, motivating the exploration of more adaptive and hybrid control designs.

Multi-agent and zone-based coordination models have recently gained attention in distributed fog and edge environments. Yang et al. propose a decentralized multi-agent scheduling framework in which IoT devices, fog nodes, and cloud nodes cooperate through local decisions and peer-to-peer negotiation to improve adaptability under uncertainty [16]. Other agent-based or peer-to-peer schedulers similarly distribute control across multiple coordinators to enhance scalability and resilience under dynamic workloads [17], [18]. While these approaches improve autonomy and fault tolerance, they may still suffer from performance degradation under workload surges and imbalanced request concentrations.

Topology-aware and latency-sensitive scheduling has also been widely studied. YuanRong is a production-grade serverless platform that integrates hierarchical scheduling and locality-aware execution to achieve low-latency invocation at large scale [19]. Other work formulates topology-aware edge service deployment problems and proposes algorithms that jointly consider spatial demand distribution and network structure to improve latency and resource utilization [20]. These studies highlight the importance of incorporating topology and locality into placement decisions.

Finally, QoS-driven adaptation and long-tail mitigation techniques leverage runtime performance signals to guide scheduling and replication. QoS-aware schedulers use latency and device capability indicators to reduce deadline violations in heterogeneous edge-cloud settings [21]. Tail-latency-oriented approaches further employ speculative execution or redundancy to mitigate unpredictable delays [22]. These methods demonstrate the effectiveness of latency and tail behavior as fine-grained control signals.

Overall, existing studies provide complementary advances in topology-aware placement, decentralized coordination, and QoS-driven adaptation. However, most systems assume a fixed control architecture—centralized, federated, or decentralized—throughout runtime, despite their known performance differences under varying workload conditions. To the best of our knowledge, no prior work integrates multi-agent control with QoS feedback to enable architecture-level adaptation, dynamically switching among scheduling modes in response to long-tail latency behavior in the edge-cloud continuum. Our work takes an initial step toward this direction.

III. SYSTEM OVERVIEW

This section presents the overall design of the proposed multi-agent scheduling framework, the supported architecture modes and request routing behavior, and the QoS-driven mechanism used to evolve the scheduling architecture at runtime.

A. Multi-Agent Architecture & Roles

The system assumes a Function-as-a-Service deployment spanning multiple edge and cloud nodes. Each node hosts a lightweight agent that implements the control logic required for scheduling and offloading, while the underlying FaaS platform (e.g., OpenFaaS) is responsible for container

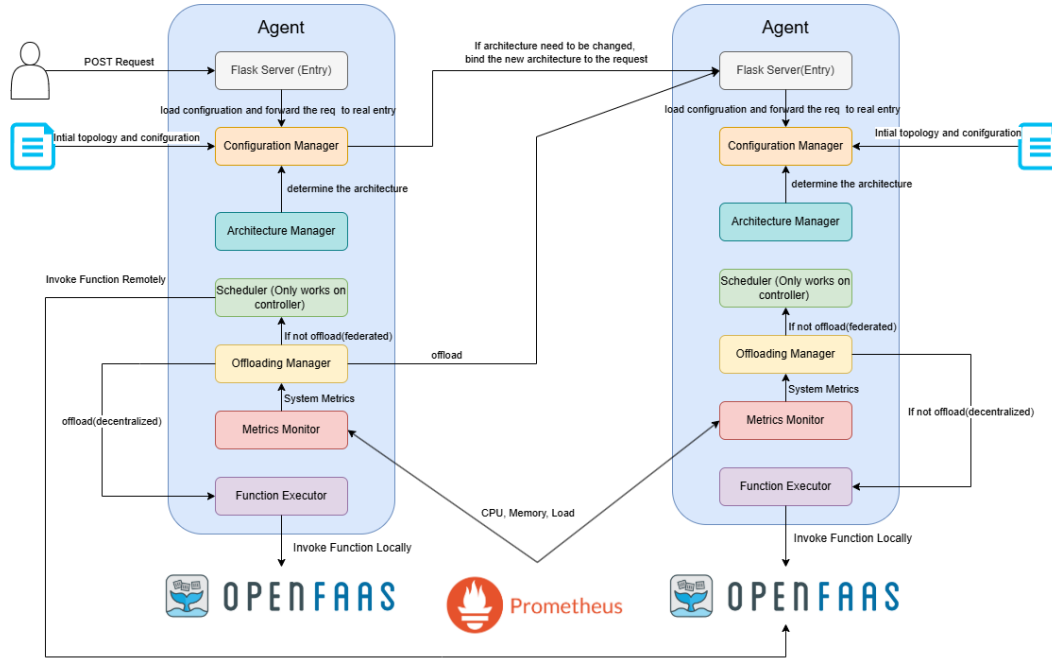


Fig. 1: Modular agent architecture supporting dynamic role activation across different architecture modes.

management and function execution. Rather than deploying separate schedulers for different coordination models, the framework consolidates all control logic into a modular multi-agent layer that can be reconfigured at runtime.

Each agent retrieves its responsibilities from a topology abstraction encoded in a local configuration file (e.g., YAML), which describes node roles (edge node, zone controller, cloud controller), zone membership, and connectivity. This human-readable and machine-parsable representation allows agents to reason about their position in the hierarchy and the boundaries of their scheduling decisions without relying on a global coordinator. Fig. 1 illustrates the modular agent architecture, where the following core components are activated based on the node's role:

- **Configuration Manager:** loads the topology and system mode, and exposes the node's role and zone information to other components.
- **Architecture Manager:** determines, for each incoming request, which scheduling architecture (centralized, federated, decentralized) should currently be applied and attaches the corresponding metadata to the request.
- **Metrics Monitor:** collects local system metrics (e.g., CPU, memory, request latencies) and exports them for QoS-driven decisions.
- **Offloading Manager:** decides whether to execute a request locally or offload it to another agent, based on the active architecture and hop-aware heuristics.
- **Scheduler:** implements per-architecture scheduling policies and is enabled only on nodes acting as zone or cloud controllers.

Roles are defined as follows. *Edge agents* are deployed

on resource-constrained edge nodes and primarily execute functions locally or forward them within the same zone. *Zone controllers* are responsible for intra-zone coordination and load balancing across a group of edge nodes. *Cloud controllers* run in the cloud and perform global coordination across zones when centralized control is active. Depending on the current architecture configuration, agents selectively enable the modules required for their role, enabling the same codebase to support different deployment scenarios and architectural modes.

By interpreting configuration locally and exchanging architecture-aware metadata with peers, agents can coordinate without a central orchestrator and reconfigure their behavior at runtime. This yields a reconfigurable multi-agent control plane that naturally supports adaptive edge-cloud operations: nodes can adjust their control responsibilities and collaboration patterns as the scheduling architecture evolves.

B. Architecture Modes & Request Flow

The framework supports three scheduling architectures that mirror common designs for edge-cloud coordination: centralized, federated, and decentralized control. In centralized mode, a global controller in the cloud is responsible for selecting execution targets across all nodes; in federated mode, zone-level controllers coordinate scheduling within their respective zones and cooperate for cross-zone offloading; in decentralized mode, each node makes independent scheduling and offloading decisions based on local information.

Fig. 2 depicts the request routing logic across these modes. All external requests enter the system at an edge agent, typically via a lightweight gateway (e.g., a Flask-based HTTP endpoint). The agent consults the *Architecture Manager*

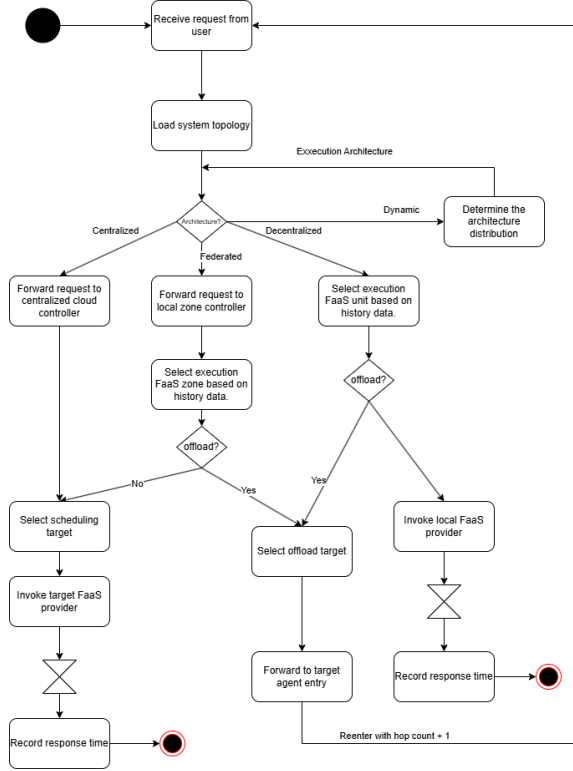


Fig. 2: Request routing logic across three control architectures.

to determine the effective architecture for the request and attaches architecture metadata (e.g., a mode label and hop count) to the request context. Based on this metadata and its role:

- in *centralized* mode, the edge agent forwards the request to the cloud controller, which selects the target FaaS node and returns a decision;
- in *federated* mode, the edge agent forwards the request to its zone controller, which chooses a node within the zone or escalates to the cloud controller if necessary;
- in *decentralized* mode, the edge agent either executes the function locally or offloads it directly to a peer node based on hop-aware heuristics.

Each forwarding step increments a hop count and reuses the same decision logic, enabling recursive delegation while avoiding cycles by enforcing a maximum hop count (e.g., at most two offloading hops before mandatory local execution). Because the architecture metadata travels with the request, agents can maintain consistent behavior during transitions: when the global architecture configuration is updated, newly arriving requests are bound to the new mode, while in-flight requests continue under the previous mode.

Configuration updates are applied through on-the-fly reloading of topology and role descriptions at each agent, without requiring system-wide synchronization or redeployment. This design effectively turns the scheduling layer into a runtime-reconfigurable operations plane: the coordination mode can be changed while the system remains online,

aligning with the goal of autonomous and dynamic operations in edge-cloud environments.

C. Tail-Aware Architecture Evolution

To decide *when* and *how* to switch between architectures, the framework employs a QoS-driven evolution mechanism based on long-tail latency behavior. Instead of relying solely on mean latency, it considers the shape of the latency distribution as an indicator of coordination pressure, system overload, and the risk of deadline violations.

For each function f and architecture mode a (centralized, federated, decentralized), the *Metrics Monitor* maintains a sliding window of recent execution times $T_{a,f}$ and derives a *tail ratio*:

$$r_{a,f} = \frac{P95(T_{a,f})}{P50(T_{a,f})}, \quad (1)$$

where $P95(\cdot)$ and $P50(\cdot)$ denote the 95th percentile and the median latency, respectively. We use $P95$ as the tail indicator because it captures QoS-relevant long-tail degradation while remaining more stable than more extreme metrics such as $P99$ or the maximum under finite sliding windows. The median $P50$ serves as the normalization baseline since it reflects typical latency and is robust to outliers; compared with the mean, it is less affected by the tail events we aim to detect. A ratio close to 1 indicates a narrow latency distribution and stable performance, whereas larger values reflect increasing skew and long-tail behavior, signaling higher risks of deadline violations and QoS degradation.

Architecture evolution is driven by comparing $r_{a,f}$ against architecture-specific *soft* and *hard* thresholds. For transitions from decentralized to federated control, the system defines two thresholds $C_{dec \rightarrow fed}^{soft}$ and $C_{dec \rightarrow fed}^{hard}$; similarly, for federated to centralized transitions it defines $C_{fed \rightarrow cen}^{soft}$ and $C_{fed \rightarrow cen}^{hard}$. These thresholds are used to compute preference weights for higher-level architectures via a piecewise-linear mapping:

$$w = \begin{cases} 0, & r_{a,f} < C^{soft}, \\ \frac{r_{a,f} - C^{soft}}{C^{hard} - C^{soft}}, & C^{soft} \leq r_{a,f} \leq C^{hard}, \\ 1, & r_{a,f} > C^{hard}. \end{cases} \quad (2)$$

In practice, the framework aggregates these weights across functions and uses them to derive a *raw* distribution over architectures (i.e., the fractions of traffic to be routed through decentralized, federated, or centralized control), as shown in Alg. 1. Let $\tilde{\rho}^{(t)} = (\tilde{\rho}_d^{(t)}, \tilde{\rho}_f^{(t)}, \tilde{\rho}_c^{(t)})$ denote this raw distribution at time t . To avoid oscillations under volatile traffic, the system applies exponential smoothing:

$$\rho^{(t)} = (1 - \alpha) \cdot \rho^{(t-1)} + \alpha \cdot \tilde{\rho}^{(t)}, \quad (3)$$

where $\rho^{(t)}$ is the smoothed architecture distribution used for decision-making.

The smoothing factor α is dynamically adjusted based on changes in the query rate ΔQPS , following a scaled sigmoid function:

$$\alpha = 0.1 + 0.8 \cdot \frac{1}{1 + \exp(-\gamma \cdot (\Delta QPS - \delta))}, \quad (4)$$

Algorithm 1 Tail-Aware Architecture Update

```

1: procedure UPDATERATIOS(fn, durations)
2:   now  $\leftarrow$  current_time()
3:   for arch in {centralized, federated, decentralized} do
4:     if enough samples and interval elapsed then
5:       p95  $\leftarrow$  percentile(durations[arch], 95)
6:       p50  $\leftarrow$  percentile(durations[arch], 50)
7:       r[arch]  $\leftarrow$  p95 / p50
8:     else
9:       r[arch]  $\leftarrow$  prev_r[arch]
10:    end if
11:  end for
12:  qps  $\leftarrow$  recent query rate
13:  if qps  $\geq$  QPS_fed then
14:    w_fed  $\leftarrow$  map_weight(r[dec], c_soft^{d2f}, c_hard^{d2f})
15:    if qps  $\geq$  QPS_cen then
16:      w_cen  $\leftarrow$  map_weight(r[fed], c_soft^{f2c}, c_hard^{f2c})
17:    else
18:      w_cen  $\leftarrow$  0
19:    end if
20:  else
21:    w_fed, w_cen  $\leftarrow$  0
22:  end if
23:   $\tilde{\rho}_c^{(t)} \leftarrow w_{cen} \cdot w_{fed}$ 
24:   $\tilde{\rho}_f^{(t)} \leftarrow w_{fed} - \tilde{\rho}_c^{(t)}$ 
25:   $\tilde{\rho}_d^{(t)} \leftarrow 1 - \tilde{\rho}_f^{(t)} - \tilde{\rho}_c^{(t)}$ 
26:  return smooth( $\tilde{\rho}_d^{(t)}$ ,  $\tilde{\rho}_f^{(t)}$ ,  $\tilde{\rho}_c^{(t)}$ )
27: end procedure

```

where $\gamma = 0.5$ controls the steepness of the transition and $\delta = 5$ defines the inflection point. Updates are applied only when sufficient latency samples are available, mitigating the impact of noise and cold-start effects. This yields a lightweight, topology-agnostic controller that can react quickly to observed QoS degradation while remaining robust to transient fluctuations.

Thresholds c_{soft} and c_{hard} were optimized via grid search on empirical data from the testbed, by minimizing the expected performance gap:

$$\text{Gap} = \mathbb{E}[\rho_c T_c + \rho_f T_f + \rho_d T_d] - \min(T_c, T_f, T_d), \quad (5)$$

where T_c , T_f , and T_d denote the observed performance (e.g., completion time) under purely centralized, federated, and decentralized architectures, respectively.

The tail-ratio-based evolution mechanism is interpretable and general. All decisions are grounded in observable latency metrics and clearly defined thresholds, making the adaptation policy easier to reason about and tune in practice. At the same time, the controller is agnostic to function semantics, application logic, and specific FaaS implementations, allowing it to be integrated with different platforms and workloads without extensive profiling or offline training. Crucially, the mechanism operates at the architecture level: it does not merely adjust placements or replica counts within a fixed control structure, but actively shifts the system between decentralized, federated, and centralized modes in response to long-tail behavior, providing a QoS-driven loop for intelligent and adaptive edge-cloud operations.

IV. IMPLEMENTATION

We implement the proposed architecture-adaptive scheduling framework on top of OpenFaaS, using a hybrid deployment that spans cloud and edge nodes. In the cloud, we run *faas-netes* on a Kubernetes cluster, while edge nodes host lightweight *faasd* instances, both exposed through the OpenFaaS Gateway's RESTful API. This configuration provides a uniform invocation interface across heterogeneous environments and allows the scheduler to treat cloud and edge platforms as interchangeable execution targets. On each node, an agent is deployed alongside the local FaaS stack as a containerized service. The agent exposes a simple HTTP endpoint for external clients and internally interacts with the OpenFaaS Gateway to invoke functions either locally or on remote nodes. Hop-aware offloading is implemented by constructing remote invocation endpoints based on the selected target node and forwarding requests via the Gateway, enabling architecture-aware offloading without modifying the underlying FaaS runtime.

The agent encapsulates all architecture-aware scheduling logic described in Section III. At startup, the *Configuration Manager* loads a topology description from a local configuration file (e.g., YAML) that specifies the node's role (edge, zone, or cloud controller), zone membership, and connectivity information. This configuration is stored in memory and can be reloaded at runtime to reflect changes in the scheduling architecture without redeploying the agents. For each incoming request, the *Architecture Manager* consults the current architecture distribution (centralized, federated, decentralized) and attaches an architecture label to the request context. Depending on its role and the selected mode, the agent then either executes the function locally, forwards the request to a zone controller, or forwards it to the cloud controller. The *Scheduler* component is only active on controller nodes and implements history-based selection of concrete FaaS units within the controller's scope, while the *Offloading Manager* handles peer selection and hop-count tracking in decentralized mode. All these behaviors are implemented via HTTP calls to the OpenFaaS Gateway, keeping the FaaS platform itself unchanged.

Performance monitoring and tail-aware decision making are implemented as a separate, loosely coupled module. Each agent periodically collects response-time samples and resource metrics from local executions and from the OpenFaaS/Prometheus stack, aggregating per-function latency distributions for each architecture mode. The *Metrics Monitor* maintains sliding windows of recent execution times and computes the tail ratio (p95/p50) used by the architecture evolution mechanism. A dedicated *TailRatioScheduler* module translates these ratios into architecture preference weights according to the piecewise-linear policy outlined in Section III-C, and updates the architecture distribution exposed by the Architecture Manager. This design keeps QoS monitoring and control policy separate from the low-level scheduling and offloading logic, and does not require any modification

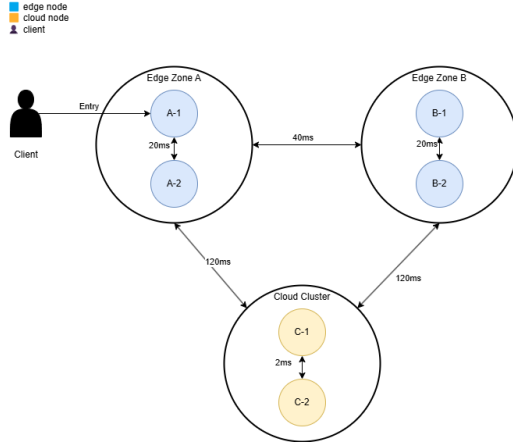


Fig. 3: Experimental edge-cloud deployment topology with emulated RTTs between zones.

to application functions: all adaptation occurs in the control plane based on observable metrics.

V. EVALUATION

This section evaluates the proposed architecture-adaptive scheduling framework along three dimensions: (i) runtime overhead of architecture selection, (ii) system-level resource footprint, and (iii) end-to-end performance under different architectures.

A. Setup and Methodology

a) Testbed.: Experiments are conducted on a cloud-edge testbed consisting of six virtual machines, reflecting a typical heterogeneous deployment across the edge-cloud continuum. The cloud domain comprises two 4-core, 8 GB nodes that jointly host the *faas-netes*-based OpenFaaS cluster, while the edge domain comprises four 2-core, 4 GB nodes running *faasd* instances. All VMs are deployed on isolated physical hosts to minimize interference. Network conditions are emulated using Linux `tc`, following the topology illustrated in Fig. 3, with distinct round-trip times (RTTs) for intra-edge, intra-cloud, inter-edge, and edge-cloud paths to approximate realistic wide-area conditions. Edge nodes act as ingress gateways for external clients, while cloud nodes only serve as execution backends.

b) Workloads & Load Generation.: Our primary benchmark is a basic matrix multiplication task implemented as an OpenFaaS function, representing a CPU-intensive workload. We additionally evaluated a data-intensive image resizing function and observed qualitatively similar architectural trends, so we omit its detailed plots for brevity. Load is generated using the Locust framework, which emulates concurrent users sending HTTP requests to a dedicated `/entry` endpoint on edge agents. Each experiment run consists of 1000 requests with a fixed peak concurrency level; we vary the number of concurrent users across runs to study scalability under increasing load. Between runs, a short cooling period is used to reduce cross-run interference (e.g., from container reuse or caching), but this period is not further analyzed.

c) Architectures & Metrics.: We compare four architectures: *centralized*, *federated*, *decentralized*, and the proposed *dynamic* architecture that evolves between them at runtime. We focus on the distributed-entry scenario, where incoming requests are distributed across all edge nodes. For each architecture, we report system-level resource usage (aggregated CPU and memory across all nodes), total completion time for each 1000-request run, and response-time statistics (average, median, and, where relevant, the p95/p50 tail ratio) to jointly assess overhead, throughput, and QoS behavior under different scheduling modes. Each configuration is repeated multiple times and averages are reported; we omit low-level parameters (e.g., Ramp-up profiles, exact sample counts) and focus on trends that are stable across repetitions.

B. Architecture Selection Overhead & Resource Footprint

We first evaluate whether dynamic architecture evolution introduces notable overhead in terms of decision latency or resource consumption.

a) Architecture selection latency.: According to our experiments, the dynamic scheduler requires on the order of 10^{-6} seconds to select an architecture for each request across all tested concurrency levels, i.e., consistently below one microsecond. This is several orders of magnitude smaller than typical network RTTs in the testbed and negligible compared to function execution times. These results indicate that the tail-ratio-based evolution mechanism and the associated preference computations impose virtually no additional delay on the critical path of request handling.

b) CPU & memory usage.: We next examine the system-level resource footprint of dynamic scheduling. Fig. 4a compares the total CPU utilization (summed over all nodes) under the four architectures. Federated scheduling shows higher CPU usage than centralized and decentralized modes due to additional zone-level controllers on edge nodes, which introduce extra scheduling and coordination overhead. The dynamic architecture tracks the CPU usage of the baselines closely and does not incur noticeable additional consumption beyond the cost of operating the same set of controllers. Similarly, Fig. 4b presents stacked memory usage per architecture and shows that dynamic scheduling maintains memory footprints comparable to static architectures, with no substantial overhead from storing metrics or architecture distributions.

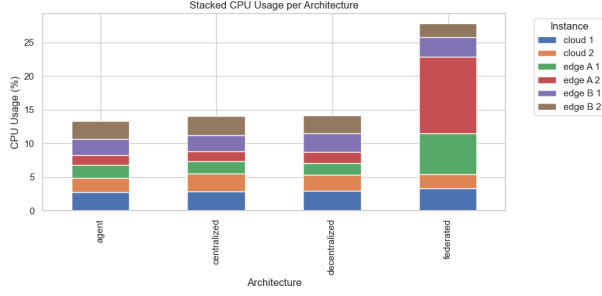
Overall, these results support the claim that dynamic, QoS-driven architecture evolution can be realized as an intelligent control layer with negligible overhead: it adds fine-grained adaptivity to the operations plane without materially increasing decision latency, CPU, or memory use compared to static architectures.

C. Performance under Different Architectures

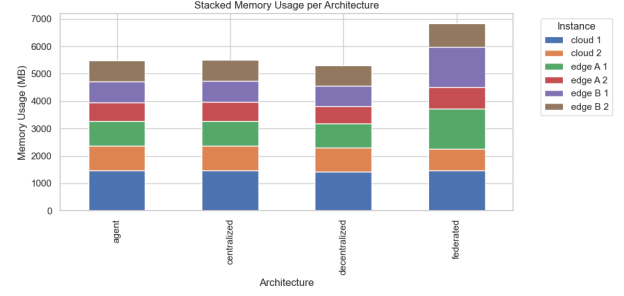
We now compare end-to-end performance across architectures, focusing on total completion time and average response time as concurrency increases.

a) Total completion time & average response time:

Fig. 5 shows the total completion time for 1000 basic (CPU-intensive) task requests under varying concurrency levels.



(a) Total CPU usage under different scheduling architectures.



(b) Stacked memory usage per scheduling architecture.

Fig. 4: System-level CPU and memory footprint across different scheduling architectures.

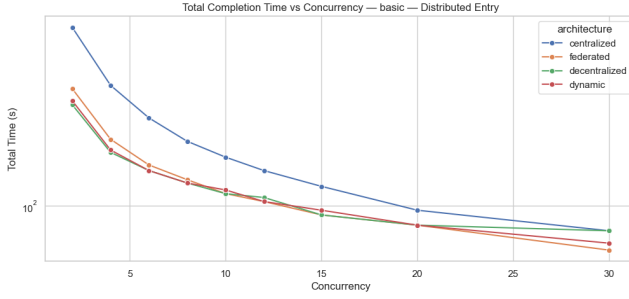


Fig. 5: Total completion time versus concurrency.

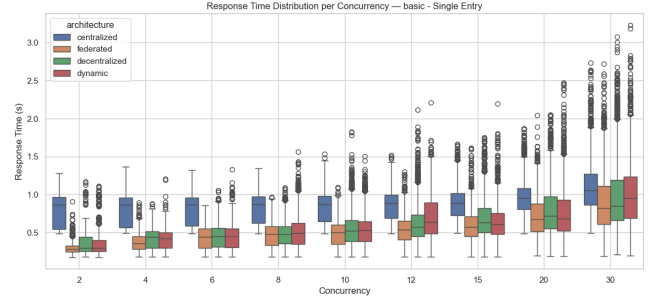


Fig. 7: Boxplot of response times.

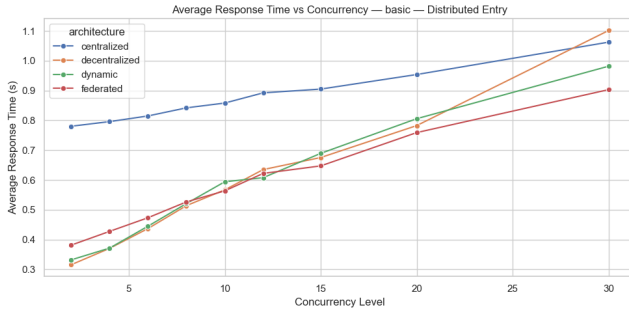


Fig. 6: Average response time versus concurrency.

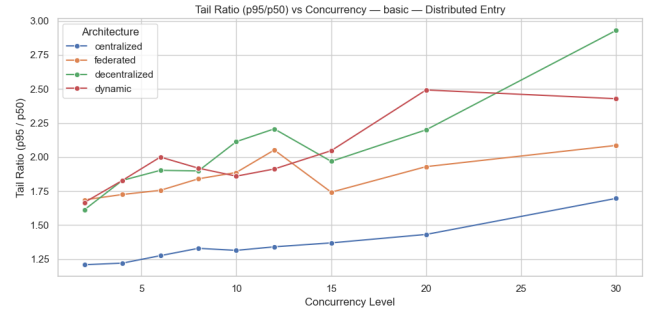


Fig. 8: Tail ratio (p95/p50) versus concurrency.

Among all architectures, the dynamic strategy achieves completion times that are consistently close to the best baseline and often yields the second-shortest total completion time, especially when the load is high. In contrast, the centralized model performs the worst because of its single-point decision bottleneck. Federated and decentralized models behave well at low concurrency, but their performance drops as the workload increases. These results indicate that dynamic scheduling can scale better by adjusting load across all FaaS units at runtime.

Fig. 6 reports the average response time under the same workload and setup. The centralized architecture shows a steady latency increase, exceeding 1.1 seconds at 30 users. The decentralized model performs well initially but fails to maintain low latency beyond 15 concurrent tasks. Federated scheduling improves over decentralized but still suffers from limited control granularity. In contrast, dynamic scheduling exhibits more sub-linear latency growth, remaining below 1 second even at the highest concurrency level, demonstrating

its ability to mitigate queuing and execution delays through architecture-aware decisions.

b) Long-tail behavior & QoS: While the above metrics focus on averages and totals, long-tail latency is often more critical for QoS in latency-sensitive applications.

The boxplots (Fig. 7) show that the centralized architecture suffers from broad latency distributions and relatively high delays even at low concurrency. Federated and decentralized architectures exhibit smaller spreads initially but deteriorate under heavier load. In contrast, the dynamic architecture sustains tight interquartile ranges with minimal high-latency outliers across all settings, indicating strong tail suppression and latency predictability. This behavior is crucial for latency-sensitive applications that demand consistent responsiveness.

The tail-ratio curves (Fig. 8) further quantify tail behavior via the ratio of 95th to 50th percentile response times. The decentralized architecture exhibits the strongest tail amplification, with tail ratios at peak load significantly higher than

those of the other modes. Centralized scheduling maintains relatively low tail ratios, largely because its baseline latency is already high and uniformly elevated. Federated control shows moderate long-tail issues. The dynamic architecture, however, maintains relatively low and stable tail ratios, illustrating its effectiveness in suppressing long-latency outliers by dynamically adjusting task placement and architecture routing.

VI. CONCLUSION

This paper presented a modular multi-agent scheduling framework for FaaS in the edge-cloud continuum that supports centralized, federated, and decentralized control and can dynamically switch between these architectures at runtime based on observed performance. The key mechanism is a tail-latency-aware evolution policy that uses simple p95/p50 ratios to detect emerging instability and drive interpretable, lightweight QoS-driven architecture selection, without modifying application functions or the underlying FaaS platform. Our OpenFaaS-based implementation and experiments on a cloud-edge testbed with emulated wide-area latencies show that the dynamic architecture reduces total completion time, stabilizes response-time distributions, and maintains comparable CPU and memory usage relative to static architectures. While tail-ratio signals can be noisy under cold-start or low-traffic conditions and our evaluation is limited to VM-based environments, the results indicate that treating the scheduling architecture itself as a runtime control variable is both practical and beneficial, and provide a basis for future work on multi-objective architecture evolution (e.g., incorporating cost, energy, and SLOs) and integration with production edge orchestration platforms.

ACKNOWLEDGMENTS

This research was partially supported by the Dutch Research Council (NWO) Large-Scale Research Infrastructures (LSRI) programme for the LTER-LIFE infrastructure (grant 184.036.014), LifeWatch ERIC, and the European Union through the projects ENVRI-Hub Next (101131141), EVERSE (101129744), BlueCloud-2026 (101094227), and OSCARS (101129751).

REFERENCES

- [1] Y. Li, Y. Lin, Y. Wang, K. Ye, and C. Xu, "Serverless computing: State-of-the-art, challenges and opportunities," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1522–1539, 2023.
- [2] G. R. Russo, V. Cardellini, and F. L. Presti, "Serverless functions in the cloud-edge continuum: Challenges and opportunities," in *2023 31st Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2023, pp. 321–328.
- [3] D. M. Botín-Sanabria, A.-S. Mihaita, R. E. Peimbert-García, M. A. Ramírez-Moreno, R. A. Ramírez-Mendoza, and J. d. J. Lozoya-Santos, "Digital twin technology challenges and applications: A comprehensive review," *Remote Sensing*, vol. 14, no. 6, 2022.
- [4] V. Mittal, S. Qi, R. Bhattacharya, X. Lyu, J. Li, S. G. Kulkarni, D. Li, J. Hwang, K. K. Ramakrishnan, and T. Wood, "Mu: An efficient, fair and responsive serverless framework for resource-constrained edge clouds," in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 168–181.
- [5] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2022.
- [6] A. Das, A. Leaf, C. A. Varela, and S. Patterson, "Skedulix: Hybrid cloud scheduling for cost-efficient execution of serverless applications," in *2020 IEEE 13th international conference on cloud computing (CLOUD)*. IEEE, 2020, pp. 609–618.
- [7] X. Cao, G. Tang, D. Guo, Y. Li, and W. Zhang, "Edge federation: Towards an integrated service provisioning model," *IEEE/ACM Transactions on Networking*, vol. 28, no. 3, pp. 1116–1129, 2020.
- [8] C. Cicconetti, M. Conti, and A. Passarella, "A decentralized framework for serverless edge computing in the internet of things," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 2166–2180, 2020.
- [9] Z. Xu, L. Zhou, W. Liang, Q. Xia, W. Xu, W. Ren, H. Ren, and P. Zhou, "Stateful serverless application placement in mec with function and state dependencies," *IEEE Transactions on Computers*, vol. 72, no. 9, pp. 2701–2716, 2023.
- [10] D. Cheng, K. Yan, X. Cai, Y. Gong, and C. Hu, "Slo-aware function placement for serverless workflows with layer-wise memory sharing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 1074–1091, 2024.
- [11] B. Zolfaghari, S. Abrishami, A. Rasoolzadegan, and B. Javadi, "Dynamic function placement and request scheduling of serverless workflows in edge environment," *IEEE Transactions on Services Computing*, vol. 18, no. 5, pp. 2781–2793, 2025.
- [12] K. Fu, W. Zhang, Q. Chen, D. Zeng, X. Peng, W. Zheng, and M. Guo, "Qos-aware and resource efficient microservice deployment in cloud-edge continuum," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 932–941.
- [13] G. R. Russo, P. Spaziani, and V. Cardellini, "Towards qos-aware serverless function offloading in the edge-cloud continuum through reinforcement learning," in *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2025, pp. 1073–1080.
- [14] A. Ellis and O. Contributors, "Openfaas - serverless functions made simple," <https://www.openfaas.com/>, 2023, accessed: 2025-06-10.
- [15] P. Daniëlse, H.-L. Tai, S. Ilager, and Z. Zhao, "Investigating efficient edge offloading architectures for serverless systems," in *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2025, pp. 1035–1038.
- [16] Y. Yang, F. Ren, and M. Zhang, "A decentralized multiagent-based task scheduling framework for handling uncertain events in fog computing," *arXiv preprint arXiv:2401.02219*, 2024.
- [17] J. Ling, T. Gupta, and A. Kumar, "Reinforcement learning for zone based multiagent pathfinding under uncertainty," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 30, 2020, pp. 551–559.
- [18] Y. Hou, Z. Wei, R. Zhang, X. Cheng, and L. Yang, "Hierarchical task offloading for vehicular fog computing based on multi-agent deep reinforcement learning," *IEEE Transactions on Wireless Communications*, vol. 23, no. 4, pp. 3074–3085, 2024.
- [19] Q. Chen, J. Qian, Y. Che, Z. Lin, J. Wang, J. Zhou, L. Song, Y. Liang, J. Wu, W. Zheng, W. Liu, L. Li, F. Liu, and K. Tan, "Yuanrong: A production general-purpose serverless system for distributed applications in the cloud," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 843–859.
- [20] G. Zou, M. Yang, S. Yang, S. Pang, S. Niu, Y. Gan, and B. Zhang, "Poibased edge service deployment with topology-aware optimization," in *2025 IEEE International Conference on Web Services (ICWS)*, 2025, pp. 720–731.
- [21] K. Sheshadri and J. Lakshmi, "Qos aware faas for heterogeneous edge-cloud continuum," in *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. IEEE, 2022, pp. 70–80.
- [22] J. Shokhanda, U. Pal, A. Kumar, S. Chattopadhyay, and A. Bhat-tacharya, "Safetail: Efficient tail latency optimization in edge service scheduling via computational redundancy management," *arXiv preprint arXiv:2408.17171*, 2024.